

Name:Kishan Patel

Reg No:25MML0030

Edge Intelligence DA-19.02.2026

Overview

This notebook demonstrates how to load a video file using Python, extract technical metadata (like resolution and frame rate) using **OpenCV**, and visualize individual frames using **Matplotlib**. It serves as a foundational script for computer vision tasks such as object detection or video classification.

1. Library Imports and Setup

The first step initializes the environment with necessary tools.

Python

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from glob import glob
```

```
import IPython.display as ipd
```

```
from tqdm.notebook import tqdm
```

- **pandas / numpy**: Standard libraries for data manipulation. NumPy is crucial here because images are stored as multi-dimensional arrays (matrices of pixel values).
 - **matplotlib.pyplot**: Used for visualization, specifically to plot the extracted video frames as static images.
 - **IPython.display (ipd)**: Allows the video to be embedded and played directly inside the Jupyter Notebook interface.
 - **cv2 (OpenCV)**: *Imported later in the notebook*. This is the core computer vision library used to decode the video file.
-

2. Video Visualization

Before processing, the notebook checks the video file by embedding a player.

Python

```
ipd.Video("026c7465-309f6d33.mp4", width=200)
```

- **Function:** ipd.Video creates an HTML5 video player within the notebook output area.
 - **Parameters:** The filename 026c7465-309f6d33.mp4 is passed, and the width is restricted to 200 pixels for a compact view.
 - **Result:** You can watch the raw video to verify the content before code analysis.
-

3. Metadata Extraction (OpenCV)

The code uses OpenCV to read technical specifications of the video file. This is essential for setting up data pipelines (e.g., resizing images to match the video dimensions).

A. Loading the Video Capture Object

Python

```
import cv2
```

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

- cv2.VideoCapture creates a connection to the video file. It does not load the entire video into memory; instead, it creates a pointer (cursor) that allows you to stream frames one by one.

B. Counting Frames

Python

```
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

- **Output:** 2398.0
- This command queries the video header to find the total number of frames.

C. Checking Resolution

Python

```
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

```
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
```

```
print(f'Height {height}, Width {width}')
```

- **Output:** Height 720.0, Width 1280.0

- The video is confirmed to be **720p HD** (1280x720 pixels).

D. Checking Frame Rate (FPS)

Python

```
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

- **Output:** FPS : 59.94
- The video runs at ~60 frames per second, which is a high frame rate standard often used in sports or smooth motion video.

E. Releasing Resources

Python

```
cap.release()
```

- This closes the connection to the file, freeing up system resources.

4. Frame Extraction

The notebook re-initializes the video capture to read the actual image data from the video stream.

Python

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read()
print(f'Returned {ret} and img of shape {img.shape}')
```

- **cap.read():** This function grabs the **next** available frame from the video.
 - **ret:** A boolean (True/False). It returns True if the frame was successfully read. If it hits the end of the video or an error occurs, it returns False.
 - **img:** The actual image data as a NumPy array.
 - **Output:** Returned True and img of shape (720, 1280, 3)
 - **720:** Height in pixels.
 - **1280:** Width in pixels.
 - **3:** Color channels (Blue, Green, Red).
-

5. Image Visualization

Finally, the notebook plots the single frame extracted in the previous step.

Python

```
plt.imshow(img)
```

- **Function:** `plt.imshow` renders the NumPy array as an image.
- **Important Note on Colors:**
 - **OpenCV** loads images in **BGR** (Blue-Green-Red) order by default.
 - **Matplotlib** displays images in **RGB** (Red-Green-Blue) order.
 - Because of this mismatch, if you display the image directly without conversion, the colors will look "off" (typically everything looks slightly blue). In a production environment, you would typically run `img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before plotting to fix the colors.

1. The Execution Crash: `AttributeError: 'NoneType'`

Where the I faces it: In the cell where you extract the frame and check its shape:

Python

```
ret, img = cap.read()
print(f'Returned {ret} and img of shape {img.shape}')
```

The Error:

If someone downloads your notebook and runs it without having the exact video file named "026c7465-309f6d33.mp4" in the exact same folder, the code will crash here.

OpenCV (`cv2.VideoCapture`) is silent when it can't find a file—it doesn't throw a "File Not Found" error. Instead, `cap.read()` just returns `None` for the image variable (`img`). When the print statement tries to look up `img.shape`, the program crashes with: `AttributeError: 'NoneType' object has no attribute 'shape'`.

How to solve it:

You should add a safeguard right after trying to open the video to check if it actually loaded successfully before proceeding to read frames.

Fixed Code:

Python

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')

# Add this check to prevent the crash

if not cap.isOpened():

    print("Error: Could not open video file. Please check the file path.")

else:

    ret, img = cap.read()

    if ret:

        print(f'Returned {ret} and img of shape {img.shape}')

    else:

        print("Error: Could not read the frame.")
```

2. The Visual Error: Incorrect Colors

Where the I faces it: In the very last cell of your notebook:

Python

```
plt.imshow(img) #opencv loads color in BGR format, matplotlib lib expects RGB format
```

The Error:

As your own comment correctly points out, the code runs, but the output is visually wrong. OpenCV stores the image array with the color channels in **Blue-Green-Red (BGR)** order. Matplotlib reads them in **Red-Green-Blue (RGB)** order. Because you didn't convert the image before plotting it, the Red and Blue channels are swapped, making the video frame look like it has a heavy blue tint or "smurf" effect.

How to solve it:

You must use OpenCV's color conversion tool to flip the channels into the correct order right before passing it to matplotlib.

Fixed Code:

Python

```
# Convert the image from BGR to RGB
```

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Plot the converted image
```

```
plt.imshow(img_rgb)
```